

Software Design Document

P2P CS2 Skin Trading Website - Initial Project Specification

Prepared for Frank Conticello
April 30, 2026

Document purpose: Define the initial business, product, technical, and operational design for a P2P CS2 skin trading website focused on safer high-value private trades.

Table of Contents

1. Executive Summary
2. Project Overview
3. Business Objectives
4. Target Personas
5. Product Scope
6. Core Features
7. Functional Requirements
8. Nonfunctional Requirements
9. System Architecture
10. Data Model
11. API Design
12. User Experience Requirements
13. Success Metrics
14. Risk Register
15. Development Milestones
16. Launch Plan
17. Open Questions
18. Acceptance Criteria
19. Conclusion

Executive Summary

This Software Design Document defines the initial product scope, system behavior, requirements, and success criteria for a peer-to-peer CS2 skin trading website focused on safer high-value private trades. The product is positioned as a trade coordination platform, not a gambling product, casino, bot marketplace, or inventory-holding escrow service.

The core product promise is simple:

Every trade includes a verifiable code that must appear in the Steam mobile confirmation. If the code does not match, the trade is unsafe.

The first market segment is 21- to 28-year-old CS2 players who trade \$200-\$1,000 weapon skins and have either been scammed, almost been API scammed, or do not fully trust direct P2P trading. The MVP is intentionally narrow: it helps two users coordinate a trade, verify the trade message, follow safety steps, and manually confirm completion.

The project should be built around trust, transparency, and operational discipline. Early success will not come from a large marketplace with liquidity. It will come from manually facilitating the first safe trades, collecting user feedback, and proving that the verification flow reduces fear during high-value P2P trades.

1. Project Overview

1.1 Product Name

Working name: **TradeGuard Protocol**

This name is temporary and may change before launch. The naming direction should remain serious, security-focused, and trust-oriented rather than gambling-oriented.

1.2 Product Vision

To become the safest lightweight coordination layer for high-value CS2 skin trades by helping users verify that the trade they approve in Steam is the same trade they initiated on the platform.

1.3 Product Positioning

The platform is not intended to operate as a casino, gambling site, or bot-powered skin exchange. The initial product should be positioned as:

- A P2P trade coordination service
- A guided trade protocol
- A verification and anti-scam layer for Steam trades
- A lightweight marketplace for manually coordinated high-value trades

1.4 Strategic Thesis

Most new skin-trading platforms fail because they attempt to solve liquidity before solving trust. This product reverses that order. The first objective is not to list thousands of items. The first objective is to help a small number of users complete trades safely and feel protected at the moment of highest fear: the Steam mobile confirmation step.

2. Business Objectives

2.1 Primary Business Objective

Build a trusted P2P trade coordination platform for CS2 skin traders who want safer high-value private trades without relying on bots or platform-held skin custody.

2.2 Secondary Objectives

1. Establish credibility within niche CS2 trading communities.
2. Prove that users will follow a guided verification protocol.
3. Facilitate the first manually verified trades before building automation.
4. Collect behavioral data about trader trust concerns, user error, and dispute patterns.
5. Build a defensible brand around safety, verification, and transparency.

2.3 Initial Market Strategy

The product should begin with a narrow user base:

- High-value private trades of \$200+
- Users who already understand CS2 skin trading
- Users who are familiar with scams, Steam trade confirmations, and API hijacking risk
- Users willing to interact directly with the founder during early beta trades

The initial go-to-market motion should be founder-led outreach in Reddit and Discord communities rather than paid advertising.

2.4 Revenue Strategy

The MVP should not prioritize monetization. Early adoption requires trust, not extraction. Recommended monetization progression:

Stage	Revenue Model	Notes
Private Alpha	0% fees	Focus on trust and feedback.
Closed Beta	Optional tips or founder-supported trades	Avoid friction while collecting data.
Public Beta	1%-2% completed-trade service fee	Only after successful trade volume exists.

Stage	Revenue Model	Notes
Growth Stage	Premium listings, verified trader badge, referral rewards	Add after core trust loop proves itself.

2.5 Legal and Compliance Positioning

The MVP should avoid gambling mechanics, randomness, case openings, house-edge systems, or platform custody of skins. The platform should not initially hold user funds or user inventory.

Important note: this document is not legal advice. Before handling payments, charging fees, expanding internationally, introducing randomized mechanics, or advertising as a marketplace, the project should be reviewed by a qualified attorney familiar with digital goods, consumer protection, gaming marketplaces, and gambling law.

3. Target Personas

3.1 Primary Persona: High-Value Private Trader

Attribute	Description
Age	21-28
Experience	Moderate to advanced CS2 trading knowledge
Trade Value	\$200-\$1,000 per trade
Current Behavior	Uses Steam, CSFloat, Discord, Reddit, or other P2P channels
Pain Point	Fear of scams, API hijacking, fake confirmations, impersonation, and trade manipulation
Motivation	Wants a safer way to complete private trades without giving custody to a bot
Trust Trigger	Seeing the exact platform-generated verification code appear in the Steam mobile confirmation

Persona Narrative

This user is comfortable with CS2 trading but still feels anxiety during high-value trades. They know that direct P2P trades can be risky, especially when trade messages, impersonators, or API-based scams are involved. They do not need a beginner tutorial. They need a system that reduces uncertainty at the exact moment they are about to accept a trade.

3.2 Secondary Persona: Mid-Tier Flipper

Attribute	Description
Age	18-30

Attribute	Description
Experience	Moderate trading experience
Trade Value	\$50-\$300 per trade
Current Behavior	Flips skins through marketplaces or P2P groups
Pain Point	Wants better margins and faster trades, but does not want to risk scams
Motivation	Reduce fee burden while maintaining basic safety
Trust Trigger	A clear, repeatable trade protocol with public accountability

3.3 Tertiary Persona: Collector

Attribute	Description
Age	20-35
Experience	Advanced niche knowledge
Trade Value	Varies widely
Current Behavior	Searches for low-float, sticker craft, souvenir, or rare items
Pain Point	Needs private negotiation and careful item verification
Motivation	Find unique items and complete safer one-off trades
Trust Trigger	Detailed trade session logs and item-specific verification notes

4. Product Scope

4.1 MVP Scope

The MVP exists to validate whether users will trust and follow a safer P2P trade process. It should include only the features required to create a trade session, generate a verification code, guide both users through the trade, and record manual confirmation.

MVP Includes

- Steam login through OpenID
- User profile with Steam ID and trade URL
- Manual trade listing creation
- Trade session creation
- Verification code generation
- Buyer/seller trade instructions

- Manual confirmation steps
- Basic trade status tracking
- Founder/admin trade oversight dashboard
- Safety warnings and error states

MVP Excludes

- Escrow
- Platform-held funds
- Platform-held skins
- Bots
- Automated trade execution
- Automated inventory syncing
- Randomized case openings
- Gambling mechanics
- On-site wallet balance
- Full payment processing
- Complex marketplace discovery
- Mobile app

4.2 Future Scope

Future phases may include:

- Reputation system
- Verified trader program
- Dispute workflow
- Trade history analytics
- Referral system
- Premium listings
- API-based inventory metadata lookup
- Automated risk checks
- Marketplace search and filtering
- Payment integration, only after legal and compliance review

5. Core Features

5.1 Steam Authentication

Users should authenticate with Steam OpenID so the platform can associate each account with a Steam identity. The system should store the Steam ID and public profile information needed for trade coordination.

Key Requirements

- Users can sign in with Steam.

- The system stores a unique internal user ID.
- The system associates the internal user ID with the Steam ID.
- Users can add or update their Steam trade URL.
- Users can view their public trade profile.

5.2 Trade Listing

A seller can create a basic trade listing for a skin they want to sell or trade.

Listing Fields

- Item name
- Item category
- Estimated value
- Asking price or desired trade value
- Float value, if applicable
- Sticker/craft notes, if applicable
- Screenshot or inspect link, if available
- Seller notes
- Listing status

5.3 Trade Session

A trade session is the core transaction object. It represents a specific buyer and seller moving through the guided trade protocol.

Trade Session States

State	Meaning
Draft	Trade has been created but not started.
Awaiting Buyer Confirmation	Buyer is reviewing details.
Awaiting Seller Action	Seller must send the Steam trade offer.
Code Verification Required	Buyer must verify that the Steam trade message contains the platform code.
Code Verified	Buyer confirmed the code matched.
Trade Completed	Both parties manually confirmed completion.
Cancelled	Trade was cancelled before completion.
Flagged	Trade requires admin review.
Disputed	One party reported a problem.

5.4 Verification Code System

The verification code system is the core product feature. Each trade session receives a unique code that the seller must include in the Steam trade message. The buyer must verify that the same code appears in the Steam mobile confirmation before accepting.

Code Format

Recommended format:

TG-####-XX

Examples:

- TG-4821-QK
- TG-9055-RX
- TG-1170-MP

Code Requirements

- Codes must be unique per active trade session.
- Codes must be easy to read on mobile.
- Codes must avoid ambiguous characters when possible.
- Codes must expire when a trade is cancelled or completed.
- Codes must be displayed prominently on the trade session page.
- The system must warn users that a missing or mismatched code means the trade is unsafe.

5.5 Guided Trade Flow

The trade page should give both parties a simple, staged process.

Seller View

The seller should see:

1. Buyer Steam profile summary
2. Item details
3. Trade verification code
4. Instructions to paste the code into the Steam trade message
5. Button: I sent the trade offer with this code
6. Safety warning: never send a trade offer without the correct code
7. Trade status timeline

Buyer View

The buyer should see:

1. Seller Steam profile summary
2. Item details
3. Expected verification code
4. Step-by-step instructions for checking the Steam mobile confirmation
5. Button: The code matches
6. Button: The code does not match

7. Warning: if the code is missing or different, do not accept the Steam trade
8. Trade status timeline

5.6 Code Mismatch Handling

If the buyer reports that the code does not match, the system should immediately move the trade into a flagged state.

System Response

- Display a high-severity warning
- Tell the buyer not to accept the Steam trade
- Pause the trade session
- Notify the seller that the code did not match
- Notify the admin/founder for manual review
- Record the mismatch event in the audit log

User Message

Recommended message:

Stop. Do not accept this Steam trade. The verification code does not match the code generated for this trade session. This may indicate an API hijack, impersonation attempt, or incorrect trade offer.

5.7 Admin Oversight Dashboard

For the earliest trades, the founder should manually oversee sessions. The admin dashboard should show active, flagged, disputed, and completed trades.

Admin Capabilities

- View all active trade sessions
- View user profiles
- View trade status history
- Mark a trade as reviewed
- Cancel a trade session
- Add internal notes
- Resolve flagged or disputed trades

6. Functional Requirements

6.1 User Authentication

ID	Requirement	Priority
FR-001	Users shall be able to authenticate using Steam OpenID.	High

ID	Requirement	Priority
FR-002	The system shall create an internal user record after successful Steam login.	High
FR-003	Users shall be able to add and update their Steam trade URL.	High
FR-004	Users shall be able to view their own profile.	Medium
FR-005	Users shall be able to view basic public profiles of trade counterparties.	Medium

6.2 Listings

ID	Requirement	Priority
FR-006	Sellers shall be able to create a manual listing.	High
FR-007	Sellers shall be able to edit their own listing before a trade session starts.	Medium
FR-008	Sellers shall be able to mark a listing as inactive.	Medium
FR-009	Users shall be able to view active listings.	Medium
FR-010	Listings shall show item name, estimated value, notes, and seller profile.	High

6.3 Trade Sessions

ID	Requirement	Priority
FR-011	Buyers shall be able to request a trade session from a listing.	High
FR-012	The system shall generate a unique verification code for each trade session.	High
FR-013	Sellers shall be instructed to include the code in the Steam trade message.	High
FR-014	Buyers shall be instructed to verify the code in the Steam mobile confirmation.	High

ID	Requirement	Priority
FR-015	Buyers shall be able to report whether the code matches.	High
FR-016	Sellers shall be able to confirm that the trade offer was sent.	High
FR-017	Both parties shall be able to manually confirm completion.	High
FR-018	Either party shall be able to cancel before completion.	Medium
FR-019	The system shall record a timestamped status history.	High

6.4 Safety and Disputes

ID	Requirement	Priority
FR-020	The system shall flag a trade when a buyer reports a code mismatch.	High
FR-021	The system shall display warnings before critical actions.	High
FR-022	The system shall store audit logs for trade events.	High
FR-023	Users shall be able to report a trade issue.	Medium
FR-024	Admins shall be able to review flagged trades.	High
FR-025	Admins shall be able to add internal notes to trade sessions.	Medium

6.5 Admin Operations

ID	Requirement	Priority
FR-026	Admins shall be able to view all active trades.	High
FR-027	Admins shall be able to filter trades by status.	Medium
FR-028	Admins shall be able to cancel or flag a trade.	High
FR-029	Admins shall be able to view user account	Medium

ID	Requirement	Priority
	details.	
FR-030	Admins shall be able to export basic trade logs.	Low

7. Nonfunctional Requirements

7.1 Security

ID	Requirement	Priority
NFR-001	The system shall enforce secure session management.	High
NFR-002	The system shall store secrets only in environment variables or managed secret stores.	High
NFR-003	The system shall validate and sanitize all user input.	High
NFR-004	The system shall protect admin routes with role-based access control.	High
NFR-005	The system shall log security-relevant trade events.	High
NFR-006	The system shall avoid storing unnecessary sensitive data.	High

7.2 Reliability

ID	Requirement	Priority
NFR-007	The system shall preserve trade session status history even if a user refreshes or returns later.	High
NFR-008	The system shall handle partial trade flows without corrupting state.	High
NFR-009	The system shall prevent duplicate active trade sessions for the same listing unless explicitly allowed.	Medium
NFR-010	The system shall provide clear recovery instructions	Medium

ID	Requirement	Priority
	when an error occurs.	

7.3 Performance

ID	Requirement	Priority
NFR-011	Trade session pages should load in under 2 seconds under normal conditions.	Medium
NFR-012	Authentication redirects should complete without unnecessary intermediate pages.	Medium
NFR-013	Admin dashboard queries should remain usable for early-stage trade volume.	Medium

7.4 Usability

ID	Requirement	Priority
NFR-014	The trade verification code shall be visually prominent on mobile and desktop.	High
NFR-015	The system shall use plain-language warnings instead of vague technical messages.	High
NFR-016	The buyer flow shall make accepting a mismatched trade feel obviously unsafe.	High
NFR-017	The UI shall minimize confusion between platform actions and Steam actions.	High

7.5 Compliance and Risk Management

ID	Requirement	Priority
NFR-018	The MVP shall not include gambling, randomized rewards, or case-opening mechanics.	High
NFR-019	The MVP shall not hold user skins.	High
NFR-020	The MVP shall not hold user funds.	High
NFR-021	Terms, disclaimers, and user	High

ID	Requirement	Priority
NFR-022	instructions shall clarify the platform's coordination role. The product shall be reviewed before introducing payment processing or gambling-like mechanics.	High

8. System Architecture

8.1 Recommended Initial Stack

Layer	Technology	Purpose
Frontend	Next.js	Web application and routing
Styling	Tailwind CSS	Fast modern UI development
Backend	Next.js API routes or Node.js/Express	Authentication callbacks, trade APIs, admin APIs
Database	Supabase Postgres	User, listing, trade session, and audit log storage
Authentication	Steam OpenID	Steam identity connection
Hosting	Vercel / free-tier cloud hosting	Initial low-cost deployment

8.2 Architecture Principles

1. Keep the MVP modular and simple.
2. Treat the trade session as the core system object.
3. Log every important event.
4. Keep safety warnings close to critical user actions.
5. Avoid automation until the manual process is proven.
6. Design database tables for auditability from day one.

8.3 High-Level Flow

1. User signs in with Steam.
2. Seller creates a listing.
3. Buyer requests a trade session.
4. System generates a verification code.
5. Seller sends the Steam trade offer manually and includes the code in the Steam message.
6. Buyer checks the Steam mobile confirmation.
7. Buyer confirms whether the code matches.
8. Both parties confirm completion.
9. System records the full status history.

9. Data Model

9.1 Core Entities

Users

Field	Type	Notes
id	UUID	Internal user ID
steam_id	String	Unique Steam identifier
display_name	String	Steam display name
avatar_url	String	Optional public avatar
trade_url	String	User-provided Steam trade URL
role	Enum	user, admin
created_at	Timestamp	Account creation date
updated_at	Timestamp	Last update date

Listings

Field	Type	Notes
id	UUID	Listing ID
seller_id	UUID	References users.id
item_name	String	Manual item name
item_category	String	Knife, rifle, glove, etc.
estimated_value	Decimal	User-provided value
asking_price	Decimal	Optional
float_value	Decimal	Optional
inspect_link	String	Optional
notes	Text	Seller notes
status	Enum	active, inactive, locked, completed
created_at	Timestamp	Listing creation
updated_at	Timestamp	Last update

Trade Sessions

Field	Type	Notes
id	UUID	Trade session ID
listing_id	UUID	References listings.id
seller_id	UUID	References users.id
buyer_id	UUID	References users.id
verification_code	String	Unique active trade code

Field	Type	Notes
status	Enum	Current trade state
seller_confirmed_sent_at	Timestamp	When seller confirmed offer sent
buyer_code_verified_at	Timestamp	When buyer confirmed code matched
completed_at	Timestamp	Completion time
cancelled_at	Timestamp	Cancellation time
flagged_at	Timestamp	Flag time
created_at	Timestamp	Session creation
updated_at	Timestamp	Last update

Trade Events

Field	Type	Notes
id	UUID	Event ID
trade_session_id	UUID	References trade_sessions.id
actor_user_id	UUID	User who triggered event
event_type	String	status_changed, code_mismatch, note_added, etc.
metadata	JSON	Additional event details
created_at	Timestamp	Event timestamp

Admin Notes

Field	Type	Notes
id	UUID	Note ID
trade_session_id	UUID	References trade_sessions.id
admin_user_id	UUID	References users.id
note	Text	Internal admin note
created_at	Timestamp	Note timestamp

10. API Design

10.1 Authentication Routes

Method	Route	Purpose
GET	/api/auth/steam	Start Steam OpenID login
GET	/api/auth/steam/	Handle Steam login

Method	Route	Purpose
	callback	callback
POST	/api/auth/logout	Log out current user
GET	/api/me	Return current authenticated user

10.2 Listing Routes

Method	Route	Purpose
GET	/api/listings	Fetch active listings
POST	/api/listings	Create a listing
GET	/api/listings/:id	Fetch listing details
PATCH	/api/listings/:id	Update listing
POST	/api/listings/:id/deactivate	Deactivate listing

10.3 Trade Session Routes

Method	Route	Purpose
POST	/api/trades	Create trade session
GET	/api/trades/:id	Fetch trade session
POST	/api/trades/:id/seller-sent	Seller confirms trade offer sent
POST	/api/trades/:id/code-matched	Buyer confirms code matched
POST	/api/trades/:id/code-mismatch	Buyer reports code mismatch
POST	/api/trades/:id/complete	Confirm trade completion
POST	/api/trades/:id/cancel	Cancel trade session

10.4 Admin Routes

Method	Route	Purpose
GET	/api/admin/trades	View all trades
GET	/api/admin/trades/:id	View trade details
POST	/api/admin/trades/:id/flag	Flag a trade
POST	/api/admin/trades/:id/resolve	Resolve flagged trade
POST	/api/admin/trades/:id/notes	Add internal note

11. User Experience Requirements

11.1 Design Tone

The product should feel serious, modern, and security-first. It should avoid casino-style visuals, spinning animations, gambling language, or hype-driven dark patterns.

Recommended tone:

- Clear
- Direct
- Calm
- Trustworthy
- Security-oriented
- Modern gaming-adjacent, but not casino-like

11.2 Critical UX Principle

The platform should never let the user forget the verification code. The code should be repeated at every critical step of the trade flow, especially before the buyer accepts anything in Steam.

11.3 Warning Design

High-risk states should use explicit language:

- “Do not accept this trade.”
- “The code does not match.”
- “This may be an unsafe trade.”
- “Cancel and contact support before continuing.”

Avoid vague messages such as:

- “Something went wrong.”
- “Verification failed.”
- “There may be an issue.”

12. Success Metrics

12.1 MVP Success Metrics

Metric	Target	Why It Matters
First completed facilitated trades	5	Proves real users will follow the process.
Manual trade completion rate	80%+	Shows the flow is understandable.

Metric	Target	Why It Matters
Code verification compliance	90%+	Measures whether users actually check the code.
Reported confusion points	Fewer than 2 per trade	Indicates UX clarity.
Serious unresolved disputes	0	Protects early reputation.

12.2 Beta Success Metrics

Metric	Target	Why It Matters
Completed trades	25-50	Shows repeatable usage.
Returning users	20%+	Indicates trust and retention.
Average trade value	\$200+	Confirms niche fit.
Founder-assisted trades	80% initially, declining over time	Shows operational learning and eventual automation potential.
Trade abandonment rate	Under 35%	Measures trust and flow efficiency.

12.3 Long-Term Success Metrics

Metric	Goal
Monthly completed trade volume	Growing month over month
Dispute rate	Under 2%
Confirmed scam incidents caused by platform flow	0
Referral-driven signups	30%+
Verified trader adoption	Increasing over time

13. Risk Register

Risk	Likelihood	Impact	Mitigation
Users do not trust a new platform	High	High	Founder-led manual onboarding and transparent trade logs.
User ignores code and accepts unsafe trade	Medium	High	Repeated warnings and confirmation gates.
Steam API limitations block automation	Medium	Medium	Do not rely on automation in MVP.

Risk	Likelihood	Impact	Mitigation
Legal or compliance uncertainty	Medium	High	Avoid gambling, custody, and payments in MVP; seek legal review before expansion.
Fake users or impersonation	Medium	High	Steam login, public profile checks, admin review.
Low marketplace liquidity	High	Medium	Start with manually coordinated private trades instead of open marketplace liquidity.
Reputation damage from first dispute	Medium	High	Founder personally monitors first 100 trades.

14. Development Milestones

14.1 Week 1: Product and Technical Foundation

- Finalize MVP flow
- Create basic wireframes
- Set up Next.js project
- Set up Supabase project
- Implement authentication foundation
- Create initial database schema

14.2 Week 2: Listings and Profiles

- Build user profile page
- Add trade URL management
- Build listing creation page
- Build listing detail page
- Add basic listing status controls

14.3 Week 3: Trade Session Flow

- Build trade session creation
- Generate verification codes
- Build seller instructions
- Build buyer instructions
- Add trade status timeline
- Add manual confirmation buttons

14.4 Week 4: Admin and Safety Layer

- Build admin trade dashboard
- Add flagged trade flow
- Add code mismatch warnings
- Add audit logging
- Test full trade walkthrough
- Prepare private alpha outreach script

15. Launch Plan

15.1 Private Alpha

The first version should not launch publicly as a full marketplace. It should launch as a private trade-assist service.

Target:

- 5-10 manually facilitated trades
- Founder present in every trade
- Feedback collected after each session
- No fees
- No payments handled by platform

15.2 Outreach Channels

- Reddit CS2 trading communities
- CS2 Discord trading servers
- Direct messages to users posting WTS or trade requests
- Build-in-public posts showing the verification flow

15.3 Founder Outreach Script

Suggested message:

Yo - quick question. Have you ever almost gotten API scammed on a trade? I am building a system that forces a verification code into the Steam trade itself so it is much easier to spot unsafe trades. I am personally walking through the first trades with users - no bots, no escrow, no gambling mechanics. Would you be open to testing the flow on a trade you are already planning?

16. Open Questions

Before implementation, the following decisions should be finalized:

1. Final product name and domain direction

2. Whether listings are public or invite-only during private alpha
3. Whether trades require both users to be authenticated before session creation
4. Whether screenshots are allowed in listings during MVP
5. How founder/admin identity is presented to early users
6. What dispute language appears in the terms and user flow
7. Whether any external payment coordination is mentioned or intentionally avoided
8. Whether Discord integration is added later for support and verification

17. Acceptance Criteria

The MVP is considered ready for private alpha when:

1. A user can sign in with Steam.
2. A seller can create a listing.
3. A buyer can request a trade session.
4. The system generates a unique verification code.
5. The seller receives clear instructions to paste the code into the Steam trade message.
6. The buyer receives clear instructions to verify the same code in Steam mobile confirmation.
7. A code mismatch moves the trade into a flagged state.
8. Both parties can manually confirm completion.
9. Admin can view active, completed, flagged, and disputed trades.
10. Trade events are logged with timestamps.
11. The system contains no escrow, bots, payments, gambling, or randomized mechanics.

18. Conclusion

This project should begin as a focused trade safety protocol, not a broad marketplace. The winning wedge is not liquidity, price discovery, or gambling-style engagement. The winning wedge is trust at the moment of confirmation.

The first product milestone is to help a small number of high-value CS2 traders complete safer private trades through a guided verification process. Once the manual process proves that users understand and trust the code-based flow, the platform can gradually expand into reputation, listings, discovery, and monetization.